서버 구현 포트폴리오

io_uring 기반 C++ 게임 서버 라이브러리 및 데모 멀티플레이 서버

저자
배진후

1 프로젝트 개요

표 1: 프로젝트 요약

항목	내용
프로젝트명	ServerCore — io_uring 기반 C++ 게임서버 라이브러리
개발 범위	범용 네트워크 라이브러리 코어 + 게임서버 애플리케이션
연동 클라이언트	C++ DirectX 11 아이소메트릭 던전 크롤러 — FactionClash 포트폴리오
프로토콜	Binary Header `[size:uint16][msgId:uint16]` + Protobuf 3 payload

1.1 io_uring 기반 서버를 직접 만든 이유

Linux 환경에서 서버를 개발하면서, 기존 epoll 기반 네트워크 라이브러리를 사용하는 대신 Linux의 최신 커널 I/O 인터페이스인 `io_uring`을 직접 활용해 네트워크 엔진을 구현해보고 싶었습니다.

epoll은 이벤트를 통지받은 뒤에도 별도의 `read`/`write` 시스템콜이 필요해, I/O마다 유저-커널 컨텍스트 스위치가 발생합니다. 반면 `io_uring`은 Submission Queue와 Completion Queue를 커널과 공유 메모리로 매핑해 시스템콜 오버헤드를 크게 줄일 수 있고, Multishot Accept/Recv를 활용하면 하나의 SQE로 여러 연결과 수신 이벤트를 처리할 수 있습니다. 이러한 특성이 게임서버의 높은 동시 접속 환경과 빈번한 I/O 패턴에서 실제로 어떤 차이를 만드는지, 직접 구현하며 확인하고 싶었습니다.

1.2 라이브러리로 분리한 이유

초기 ServerCore는 라이브러리 계층(`IoRing`, `Session`, `BufferPool`)과 프레임워크 계층(`IoLoop`, `GameRoom`, `PacketSession`)이 한 레이어에 혼재되어 있었습니다. `PlayerId`, `RoomId` 같은 게임 전용 타입까지 코어에 포함되어 있어, HTTP 서버나 DB 프록시처럼 게임과 무관한 서버를 만들 때도 불필요한 게임 의존성을 함께 가져와야 했습니다.

이 문제를 해결하기 위해 코어에서 게임 프레임워크 코드를 분리했습니다. `IoLoop`, `GameRoom`, `PacketSession`, `IoCommand` 등 성격이 라이브러리 성격과 맞지 않는 클래스와 타입을 프레임워크 계층으로 옮겼습니다. servercore 라이브러리는 네트워크 I/O, 세션 관리, 버퍼 관리, 이벤트 디스패치 같은 범용 기능만 제공하는 순수한 라이브러리로 재구성했고, 게임서버는 라이브러리 위에 자체 프레임워크를 구현하는 하나의 애플리케이션으로 구현했습니다.

1.3 코어 라이브러리 구조

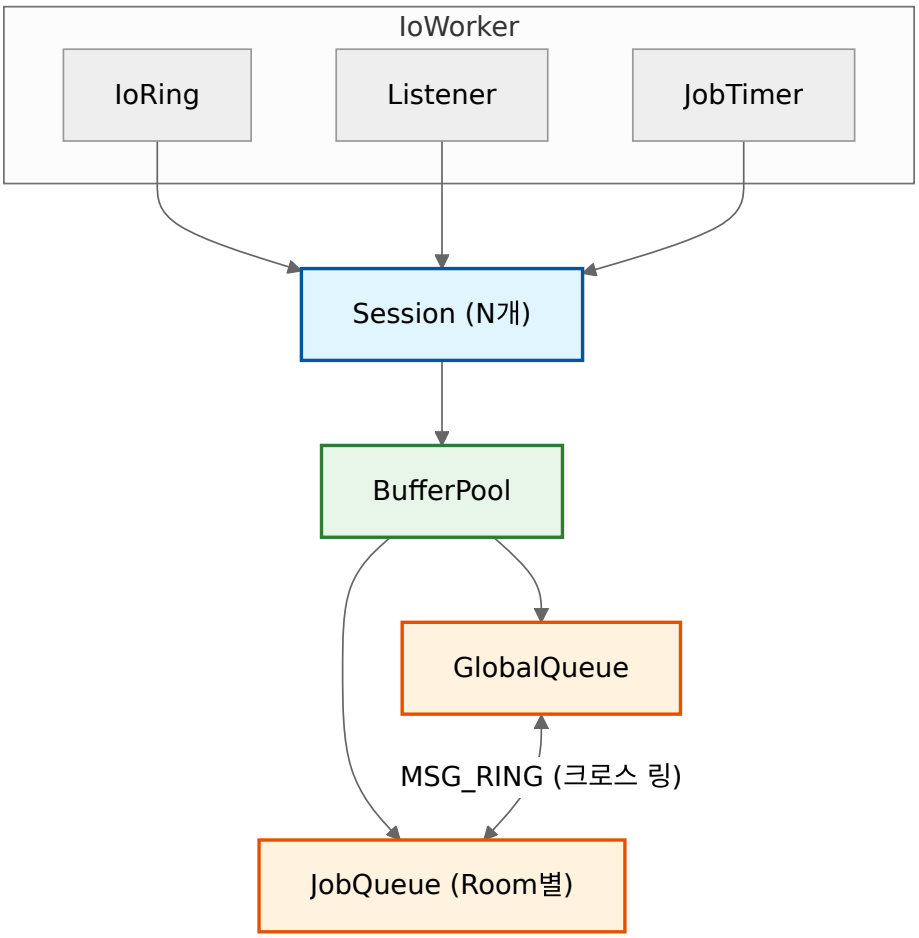

그림 1: ServerCore v4 코어 아키텍처

2 io_uring 네트워크 엔진

ServerCore의 네트워크 엔진은 Linux `io_uring` 을 기반으로 합니다. 커널과 유저스페이스가 Submission Queue(SQ)와 Completion Queue(CQ)를 공유 메모리로 매핑하므로, I/O 요청 제출과 완료를 최소한의 시스템콜로 처리할 수 있습니다.

`IoRing` 클래스는 `io_uring` 인스턴스를 RAII로 관리하며, 현재 스레드의 링을 참조합니다. 핵심 기능은 다음과 같습니다.

- **Multishot Accept** — 하나의 SQE로 여러 연결을 수락합니다. 새 연결마다 CQE만 추가로 발생하므로 SQE를 다시 제출할 필요가 없어 accept 오버헤드를 줄일 수 있습니다.
- **Multishot Recv + Provided Buffer** — `IOSQE_BUFFER_SELECT` 플래그를 사용해 커널이 Provided Buffer Ring에서 버퍼를 선택하고, 해당 버퍼에 수신 데이터를 직접 기록합니다. 수신 경로의 버퍼 관리 비용과 복사 비용을 줄일 수 있습니다.
- **MSG_RING** — `IORING_OP_MSG_RING` 으로 다른 링에 태스크를 직접 전달합니다.

2.1 Dispatch() — CQE 배치 처리

`Dispatch()` 는 CQ에 쌓인 완료 이벤트를 배치 단위로 dispatch해 처리합니다. `user_data` 의 bit 0을 이용해 MSG_RING 태스크와 일반 I/O 이벤트를 구분합니다.

```
IoRing.cpp — Dispatch(): CQE 루프
```

```
bool IoRing::Dispatch(std::chrono::milliseconds timeout) {
    std::vector<EventHandlerRef> keep_alive;
    unsigned head = 0; unsigned count = 0;
    io_uring_for_each_cqe(ring_, head, cqe) {
        const std::uint64_t data = cqe->user_data;
        if (data & 0x1ULL) {  // MSG_RING 태스크 (bit 0 플래그)
            auto* task = reinterpret_cast<std::move_only_function<void()>*>(
                data & ~0x1ULL);
            (*task)(); delete task;
        } else if (data != 0) {  // 일반 I/O 이벤트
            auto* ev = reinterpret_cast<IoEvent*>(data);
            auto owner = ev->Owner();
            if (owner) {
                keep_alive.push_back(owner);  // 배치 중 소멸 방지
                owner->Dispatch(ev, result, flags);
            }
        }
        ++count;
    }
    io_uring_cq_advance(ring_, count);
    return true;
}
```

MSG_RING 태스크는 힙에 할당된 `move_only_function`의 주소에 bit 0을 세팅해 `user_data`에 인코딩합니다. 일반 I/O 이벤트는 `IoEvent*`를 통해 소유 핸들러로 디스패치되며, `keep_alive` 벡터는 배치 처리 중 핸들러가 먼저 파괴되는 일을 막습니다.

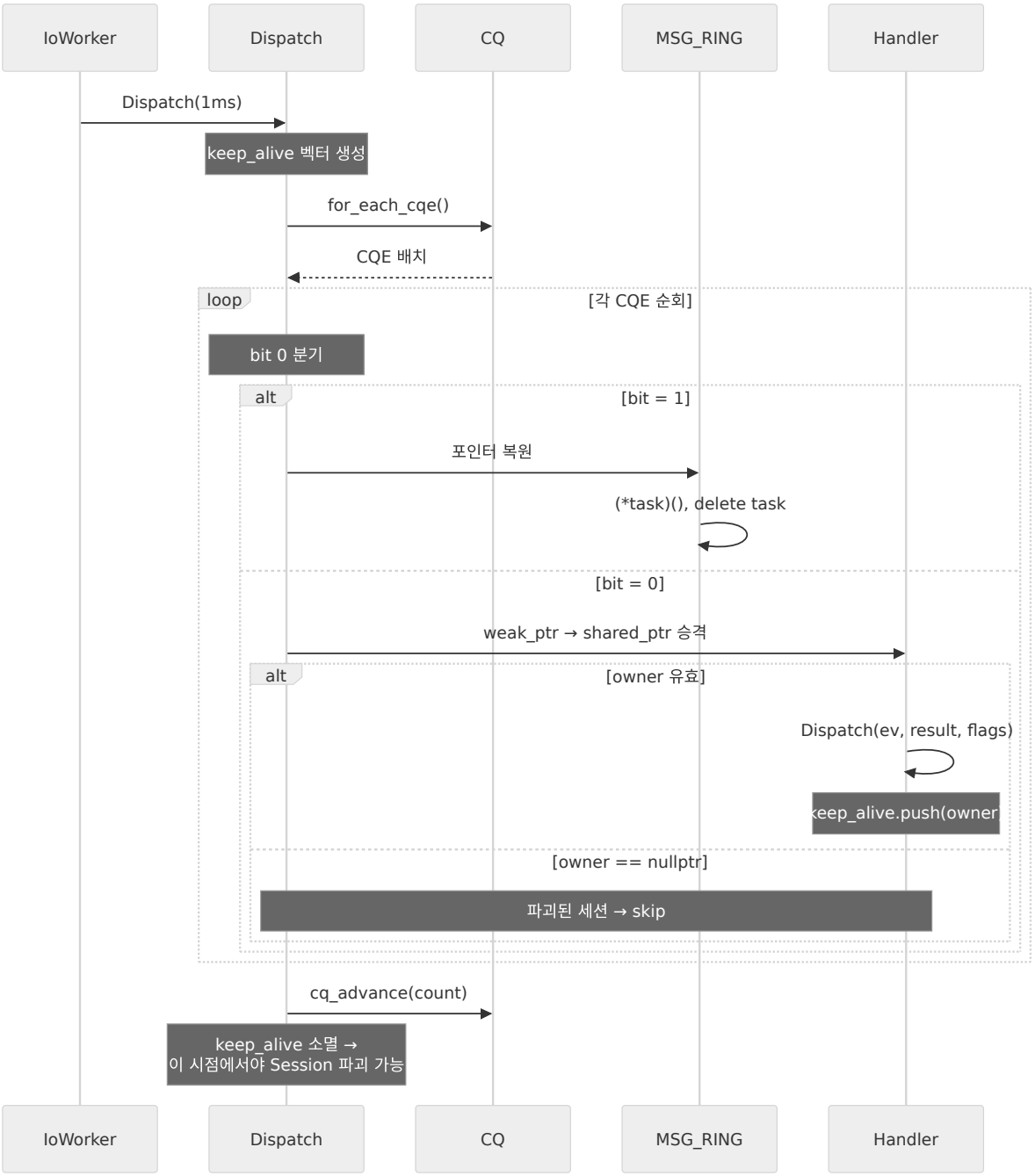

그림 2: Dispatch() 전체 흐름

2.2 Provided Buffer Ring

Provided Buffer Ring은 단일 `mmap`으로 메타데이터(`io_uring_buf` 배열)와 실제 데이터 영역을 연속으로 할당합니다.

- `RingBuffer::Create()`는 `meta_size + data_size`를 한 번에 `mmap`하여 TLB 미스를 줄입니다.
- 초기화 시 모든 버퍼를 `io_uring_buf_ring_add`로 커널 링에 등록합니다.
- 커널은 이 링에서 사용 가능한 버퍼를 직접 선택해 recv 데이터를 기록하므로, 유저스페이스에서 수신 버퍼를 매번 지정하지 않아도 됩니다.

2.3 버퍼 고갈과 패킷 단편화

Provided Buffer Ring을 실제 트래픽에 붙이면서 두 가지 문제가 드러났습니다.

첫 번째는 **버퍼 고갈(ENOBUFS)** 입니다. 동시 접속이 몰려 provided buffer가 전부 소진되면 커널이 `ENOBUFS`를 반환하며 Multishot Recv가 자동 해제됩니다. 이 상태를 별도로 감지하지 않으면 해당 세션의 수신이 조용히 멈추기 때문에, CQE 플래그를 확인한 뒤 `RegisterRecv()`로 Multishot Recv를 재등록하는 복구 경로가 필요했습니다.

두 번째는 **패킷 단편화**입니다. TCP 스트림 특성상 하나의 게임 패킷이 여러 CQE에 걸쳐 도착하는데, provided buffer는 CQE 단위로 잘려 나오므로 단편화된 패킷을 zero-copy로 바로 조립할 수 없습니다. 이 경우에는 별도의 재조립 버퍼가 필요합니다.

`Session::OnRecv()`에서는 이 두 경우를 Fast Path / Slow Path로 분리해 처리합니다. 완전한 패킷이 하나의 CQE에 담겨 오는 일반 경로(Fast Path)에서는 `F_MORE` 플래그를 확인해 `--pending_io_`를 갱신하고, 커널 버퍼를 `span`으로 직접 참조해 파싱한 뒤 즉시 `Return()`으로 반납합니다. 패킷이 단편화된 경우(Slow Path)에는 서브클래스 `PacketSession`이 별도의 `RecvBuffer`에 복사해 완전한 패킷이 모일 때까지 재조립하고, `ENOBUFS` 복구는 `RegisterRecv()`를 다시 호출해 Multishot Recv를 재등록하는 방식으로 처리합니다. 수신 직후 버퍼를 반납해 커널 버퍼 점유 시간을 짧게 유지하면, 특정 세션의 지연이 다른 세션의 버퍼 고갈로 이어지는 상황을 줄일 수 있습니다.

3 세션 수명관리

3.1 self_ref_ + pending_io_ 패턴

클라이언트 대량 접속·해제를 반복하는 스트레스 테스트에서 SEGFAULT가 간헐적으로 발생했습니다. 크래시 지점은 항상 CQE 콜백에서 `Session` 멤버에 접근하는 시점이었지만, 재현 빈도가 불안정해 원인 특정에 시간이 걸렸습니다.

원인은 **세션 수명과 커널 I/O 완료 타이밍의 불일치**였습니다. 클라이언트가 연결을 끊으면 애플리케이션은 `Session` 객체를 즉시 파괴하지만, 커널에는 아직 완료되지 않은 SQE(Multishot Recv, Send 등)가 남아 있습니다. 이후 커널이 해당 SQE의 CQE를 발행하면 **이미 해제된 메모리에 커널이 데이터를 덮어쓰는 Use-After-Free**가 발생합니다. 해제된 영역이 아직 다른 객체로 재할당되지 않은 상태에서는 조용히 지나가고, 재할당된 뒤에야 크래시로 드러나기 때문에 재현이 불규칙했던 것입니다.

해결의 핵심은 **세션 파괴 시점을 애플리케이션이 아니라 커널의 I/O 완료 신호가 결정하도록 구조를 뒤집는 것**이었습니다. 이를 위해 `Session`에 두 개의 멤버를 추가했습니다.

include/servercore/io/Session.h

```
// Counts in-flight io_uring ops whose CQEs reference Session members.
// self_ref_ must not be released until this reaches zero.
// Accessed from I/O thread only — no synchronization needed.
int pending_io_ = 0;

// Self-ownership: keeps Session alive during active I/O.
// Released only via TryRelease() after all in-flight ops complete.
std::shared_ptr<Session> self_ref_;
```

- `self_ref_` — 세션이 자기 자신을 소유하는 `shared_ptr`입니다. 외부에서 모든 참조를 놓더라도 `self_ref_`가 남아 있는 동안 세션은 유지됩니다.
- `pending_io_` — 커널에 제출됐지만 아직 CQE를 받지 못한 SQE의 수입니다. SQE 제출 시 `++pending_io_`, CQE 수신 시 `--pending_io_`를 수행합니다.

세션의 실제 소멸 시점은 애플리케이션이 아니라 커널의 완료 신호에 의해 결정됩니다. `TryRelease()`는 `pending_io_ > 0`이면 그대로 반환하고, `pending_io_`가 0이 되는 순간 — 즉 커널이 모든 in-flight I/O를 완료한 시점 — `OnDisconnected()` 콜백을 호출한 뒤 `self_ref_.reset()`으로 자기 소유를 해제합니다. 이렇게 수명의 최종 결정권을 커널 쪽에 넘겨두면, 애플리케이션 타임라인과 커널 타임라인이 어긋나 발생하던 UAF가 원리적으로 생길 수 없게 됩니다.

`pending_io_`가 0이 되지 않으면 세션이 영원히 살아남아 누적되는 문제를 해결하기 위해 `Disconnect()`는 단순 cancel이 아니라 `shutdown(SHUT_RDWR)`을 SQE로 제출합니다. 커널이 소켓을 shutdown하면 Multishot Recv는 EOF(`res == 0`)로 반드시 종료 CQE를 발행하고, in-flight Send도 에러 CQE로 즉시 종료되므로, 모든 SQE가 유한 시간 안에 회수됩니다.

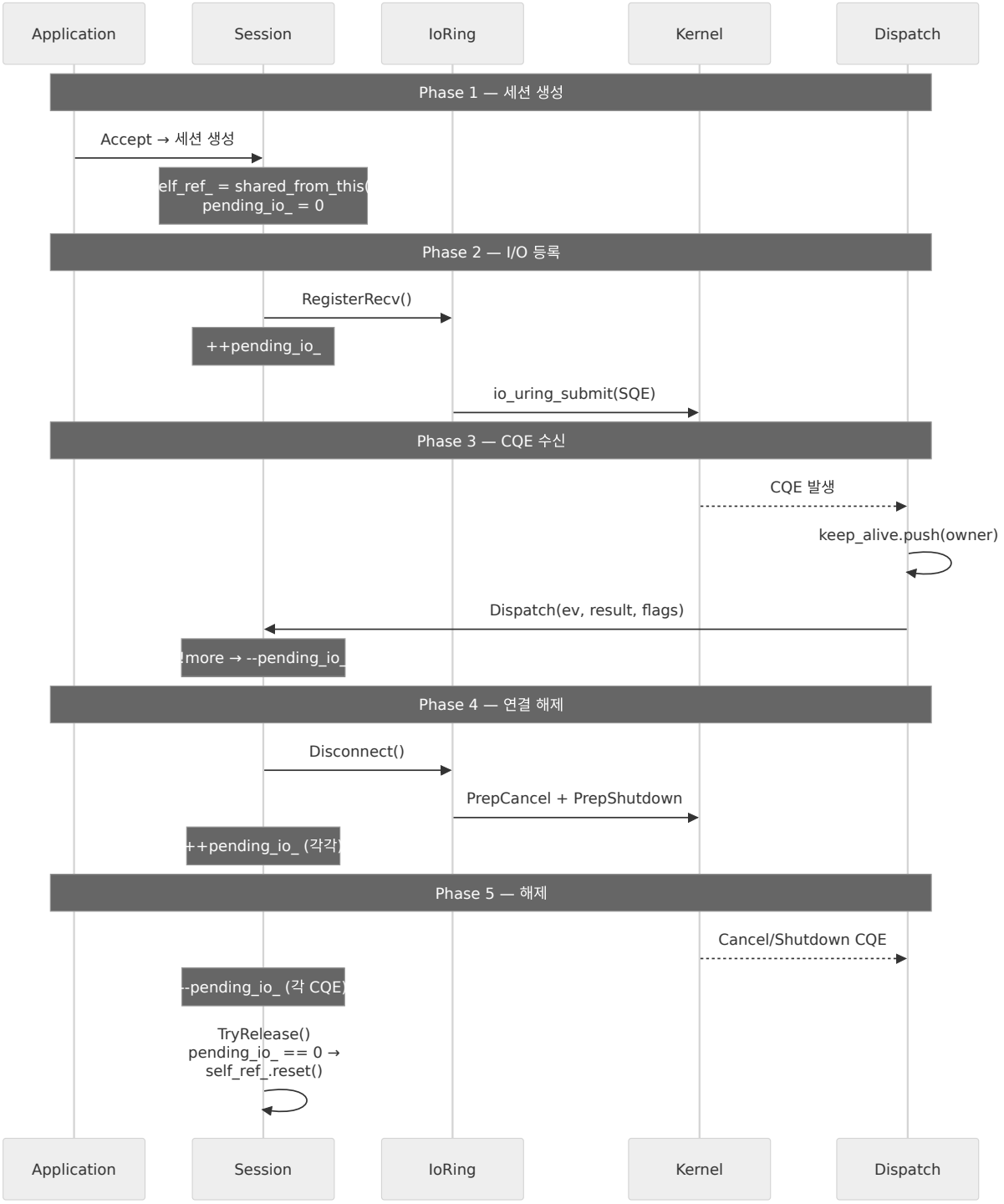

그림 3: 세션 수명관리 전체 흐름

3.2 CQE 배치 처리와 소멸 지연

앞 절의 `pending_io_` 메커니즘만으로는 부족한 경계 케이스가 하나 더 있었습니다. `Dispatch()`는 성능을 위해 여러 CQE를 한 번에 배치로 가져와 순회하는데, 이 배치 안에는 **동일 세션의 이벤트가 여러 개 섞여 있을 수 있습니다**. 앞쪽 이벤트를 처리하는 과정에서 해당 세션이 `TryRelease()`로 파괴 조건을 만족하면, 같은 배치의 뒤쪽 이벤트가 이미 소멸한 객체를 참조하는 UAF가 다시 발생합니다.

해결책은 배치 순회 동안 참조 카운트를 임시로 +1 유지하는 것입니다. CQE마다 `IoEvent::Owner()` 로 `weak_ptr` 를 `shared_ptr` 로 승격해 로컬 `keep_alive` 벡터에 보관하면, 처리 중간에 `self_ref_.reset()` 이 호출되더라도 세션은 배치가 끝날 때까지 살아 있습니다. 배치 순회가 완료되고 `keep_alive` 가 스코프를 벗어나는 시점에서야 실제 소멸이 일어나므로, 뒤쪽 이벤트도 항상 유효한 객체를 참조합니다.

3.3 IoEvent의 weak_ptr 참조

`pending_io_` 메커니즘과 `keep_alive` 배치 보호까지 갖춘 뒤에도, 한 가지 시나리오에서는 세션이 **논리적으로 죽었지만 물리적으로 살아남는 구간**이 발생했습니다. Heartbeat 타임아웃이나 강제 킥처럼 커널 I/O가 끝나기 전에 세션을 끊어야 하는 상황에서, 만약 `IoEvent` 가 소유자를 `shared_ptr` 로 붙잡고 있으면 in-flight I/O가 전부 완료될 때까지 세션 객체가 불필요하게 유지됩니다.

이를 피하기 위해 `IoEvent` 는 소유자를 `weak_ptr` 로만 참조합니다. CQE 처리 시 `Owner()` 에서 `weak_ptr` → `shared_ptr` 승격을 시도해 `nullptr` 가 반환되면 해당 이벤트는 건너뜁니다. 이벤트 객체가 세션의 수명을 연장하지 않으므로, 세션이 논리적으로 종료된 뒤에 도착하는 CQE도 자연스럽게 무시됩니다.

4 멀티스레드 아키텍처

4.1 Ring-per-thread와 세션 종속 문제

io_uring은 ring-per-thread 모델로 동작합니다. Multishot Accept로 연결을 수락하면 해당 세션은 Accept를 수행한 링에 종속되고, 이후의 Recv/Send도 모두 그 링에서 처리됩니다. 여기서 두 가지 문제가 파생됩니다. 세션이 accept된 링에 고정되기 때문에 특정 워커에 연결이 몰리면 부하가 편중되고, Room이나 Match처럼 여러 세션에 걸친 로직에서 다른 링에 있는 세션으로 데이터를 보내려면 별도의 크로스 링 통신이 필요합니다.

부하 편중은 `SO_REUSEPORT` 를 이용했습니다. 각 `IoWorker` 가 동일한 포트에 자체 `Listener` 를 바인드해두면 커널이 `accept()` 를 워커 간에 분배하므로, 연결 단위로 자연스럽게 나눠집니다.

크로스 링 통신은 `IORING_OP_MSG_RING` opcode를 씁니다. 다른 링의 CQ에 태스크를 커널 경로로 직접 삽입하는 방식이라 유저스페이스 뮤텍스나 이벤트 fd 같은 중간 단계가 필요 없고, 세션을 다른 링으로 마이그레이션하지 않고도 Room 브로드캐스트 같은 크로스 링 작업을 처리할 수 있습니다.

4.2 SO_REUSEPORT — OS 레벨 부하 분산

`IoWorker::Start()` 는 링 생성 후 `SO_REUSEPORT` 로 바인드된 `Listener` 를 만들고, 독립 스레드에서 이벤트 루프를 구동합니다.

4.3 MSG_RING — 링 간 제로카피 태스크 전달

`RunOnRing()` 은 태스크 포인터의 bit 0을 플래그로 인코딩해 대상 링으로 전달합니다. 같은 링이면 즉시 실행하고, 다른 링이면 `IORING_OP_MSG_RING` SQE를 제출합니다. `IOSQE_CQE_SKIP_SUCCESS` 를 사용해 소스 링에서 불필요한 완료 CQE가 생성되지 않도록 했습니다.

IoRing.cpp — RunOnRing(): MSG_RING 태스크 전달

```
bool IoRing::RunOnRing(std::move_only_function<void()> task) noexcept {
    if (t_current_ == this) { task(); return true; }  // 같은 링이면 즉시 실행
    if (t_current_ && t_current_->ring_) {
        auto* task_ptr = new std::move_only_function<void()>(std::move(task));
        std::uint64_t tagged = reinterpret_cast<std::uint64_t>(task_ptr) | 0x1ULL;
        io_uring_sqe* sqe = io_uring_get_sqe(t_current_->ring_);
        io_uring_prep_msg_ring(sqe, Fd(), 0, tagged, 0);
        sqe->flags |= IOSQE_CQE_SKIP_SUCCESS;  // 소스 링 CQE 억제
        io_uring_submit(t_current_->ring_);
```

```
        return true;
    }
    Post(std::move(task)); return true;
}
```

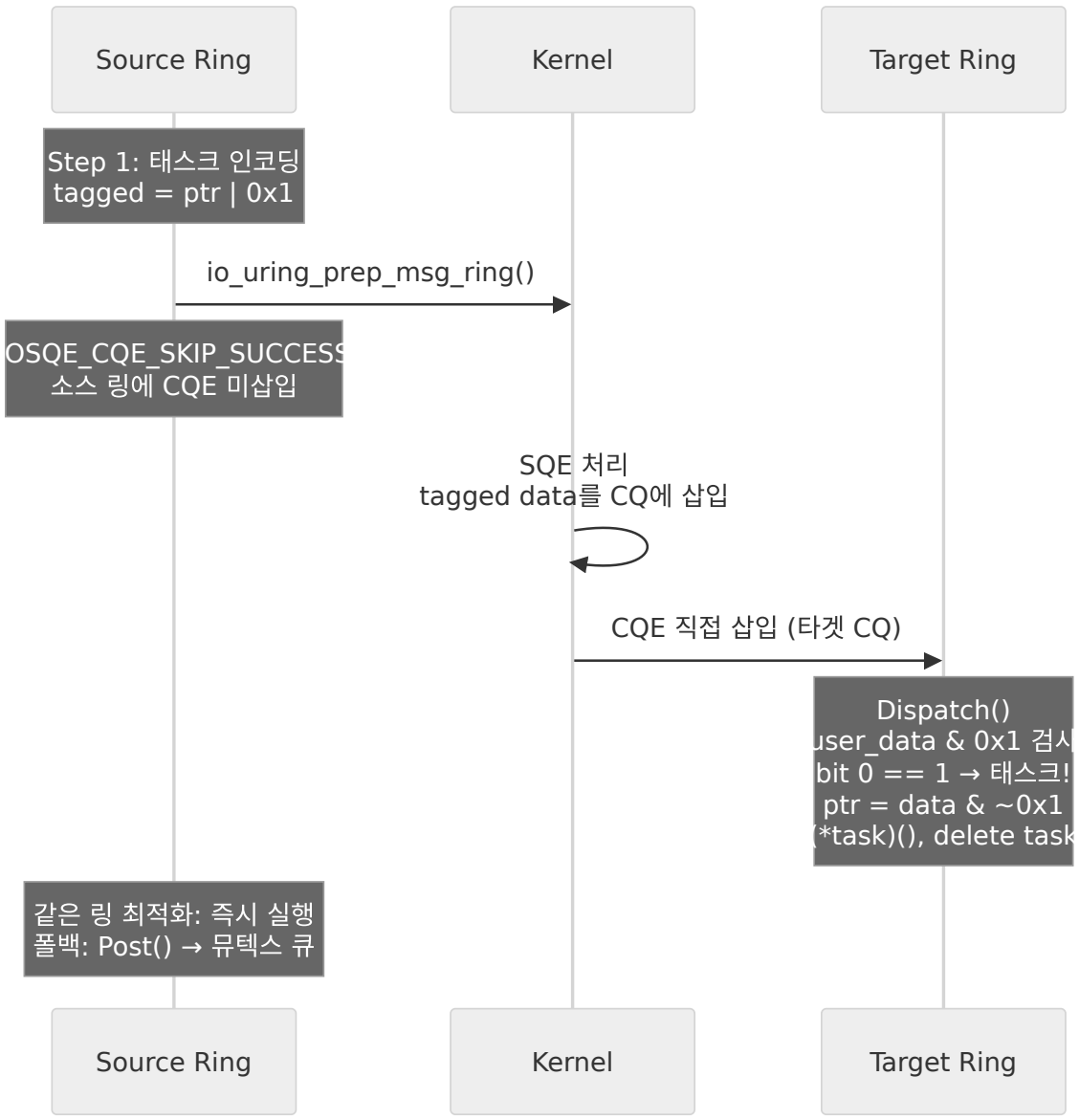

그림 4: MSG_RING 크로스 링 통신 흐름

5 대칭형 이벤트 루프 — IoWorker

모든 `IoWorker`는 I/O와 게임 로직을 동일한 단일 루프에서 실행합니다. 별도의 게임 로직 전용 스레드를 두지 않아 컨텍스트 스위치를 줄였고, 8ms 실행 예산을 둬 하나의 Room이 한 틱 동안 I/O를 독점하지 않도록 했습니다.

```
io_worker.cpp — Run(): IO + 게임 로직 단일 루프
```

```
void IoWorker::Run() {
    IoRing::SetCurrent(ring_.get());
    while (running_) {
        ring_->Dispatch(std::chrono::milliseconds(1));     // 1. I/O 이벤트
        ring_->ProcessPostedTasks();                        // 2. 크로스 스레드 태스크
        timer_.DistributeExpired();                         // 3. 타이머 만료 작업
        while (auto* q = global_queue_.TryPop())
            q->Execute(std::chrono::steady_clock::now()
                       + std::chrono::milliseconds(8));     // 4. 게임 로직 (8ms 예산)
```

```
    }
}
```

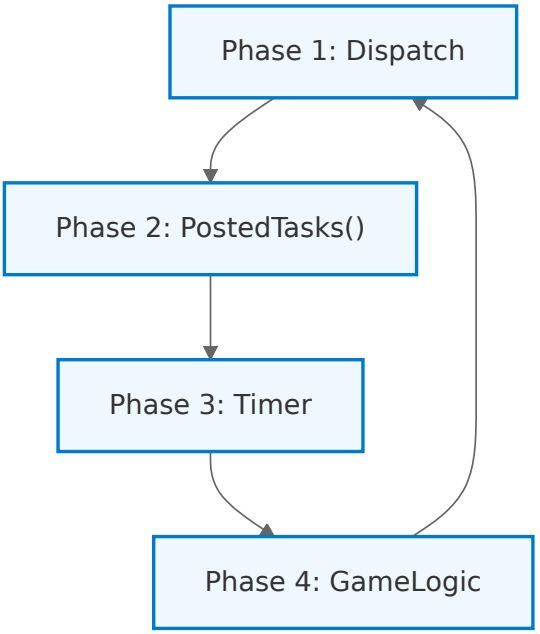

그림 5: IoWorker 대칭형 이벤트 루프

표 2: IoWorker 이벤트 루프 단계

단계	처리 대상	함수 / 설명
Phase 1	I/O 이벤트	`ring_->Dispatch()` — CQE를 배치로 수확하고 처리
Phase 2	크로스 스레드 태스크	`ProcessPostedTasks()` — 다른 스레드에서 넘어온 태스크 실행
Phase 3	타이머	`timer_.DistributeExpired()` — 만료된 작업 분배
Phase 4	게임 로직	`GlobalQueue`에서 `JobQueue`를 꺼내 8ms 예산 내에서 실행

5.1 JobQueue & 스케줄링

게임서버에서 Room 하나는 하나의 독립적인 게임 월드입니다. 여러 Room이 소수의 `IoWorker` 스레드를 공유하기 때문에, 특정 Room의 로직이 오래 걸리면 같은 스레드에 배정된 다른 Room의 틱이 밀리는 문제가 발생합니다. 이를 해결하기 위해 `JobQueue` + `GlobalQueue` 구조로 스케줄링을 구현했습니다.

5.1.1 JobQueue — 락-프리 직렬 실행

`JobQueue`는 Room·Match 등 게임 오브젝트가 상속하는 직렬 실행 큐입니다. 외부 스레드에서 `Push(job)`를 호출하면 큐에 적재되고, 실제 실행은 단일 스레드에서만 이루어지므로 게임 로직 내부에서는 뮤텍스가 필요하지 않습니다.

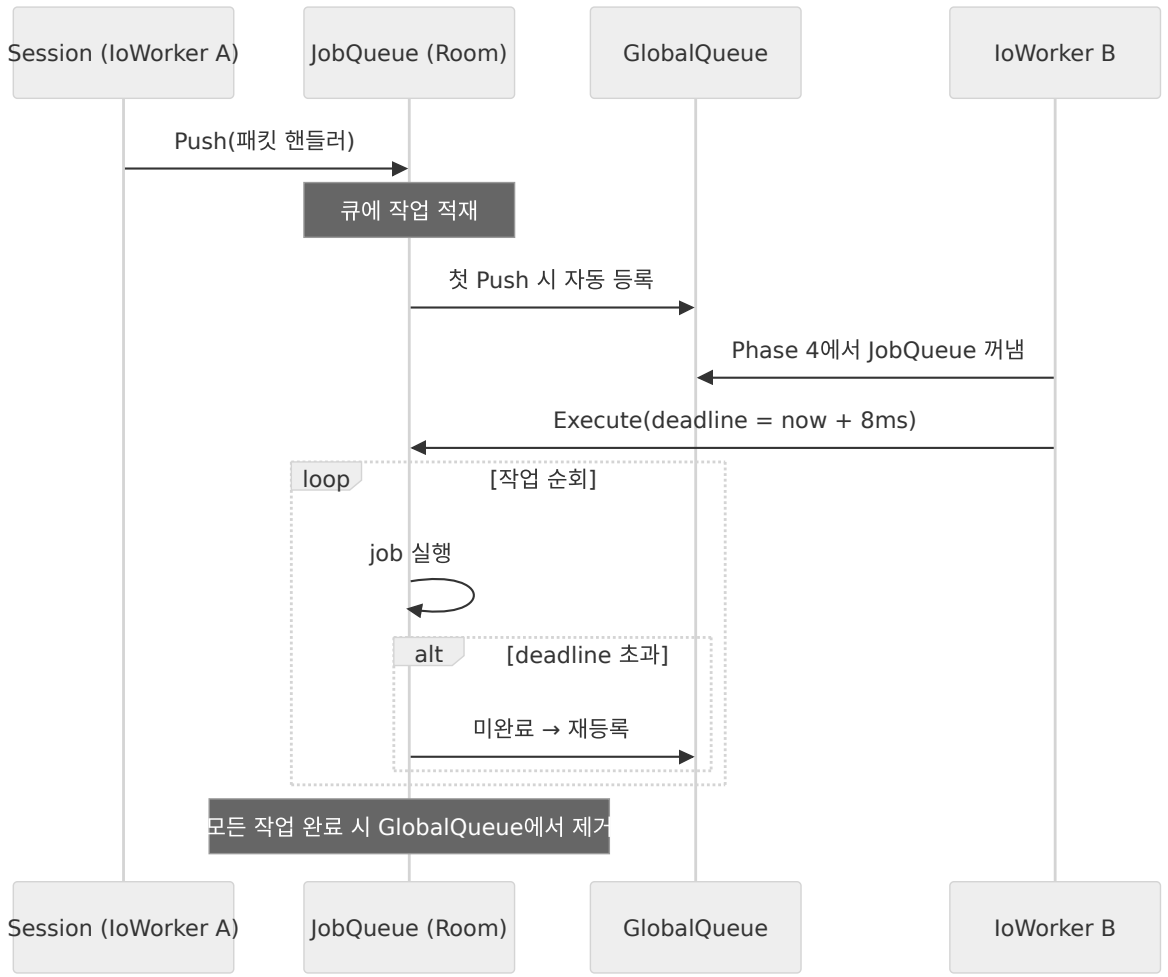

그림 6: JobQueue 실행 사이클

실행 흐름의 핵심 세 단계:

1. **Batch Swap** — 뮤텍스를 잡고, 현재 큐 전체를 로컬 `batch` 변수와 교환한 뒤 즉시 해제합니다. 이후 순회 중에는 락을 잡지 않으므로, Push와 Execute가 동시에 진행되어도 경합이 최소화됩니다.

2. **시간 예산 실행** — `Execute(deadline)` 은 `steady_clock::now()` 가 deadline(8ms)을 넘지 않는 동안만 작업을 꺼내 실행합니다. 8ms는 60fps 기준 한 프레임(16.6ms)의 절반으로, 나머지 시간을 I/O 처리에 할당하기 위한 값입니다.

3. **이어 실행** — 예산 초과로 남은 작업이 있으면 큐 앞에 복원하고, `JobQueue` 자신을 다시 `GlobalQueue` 에 등록합니다. 다음 Phase 4에서 이어서 실행되므로 작업이 유실되지 않습니다.

5.1.2 GlobalQueue — 워크 스틸링 분배

`GlobalQueue` 는 실행 대기 중인 `JobQueue` 들의 중앙 저장소입니다. 여러 `IoWorker` 가 Phase 4에서 경쟁적으로 `JobQueue` 를 꺼내 실행하는 **워크 스틸링** 패턴으로 동작합니다.

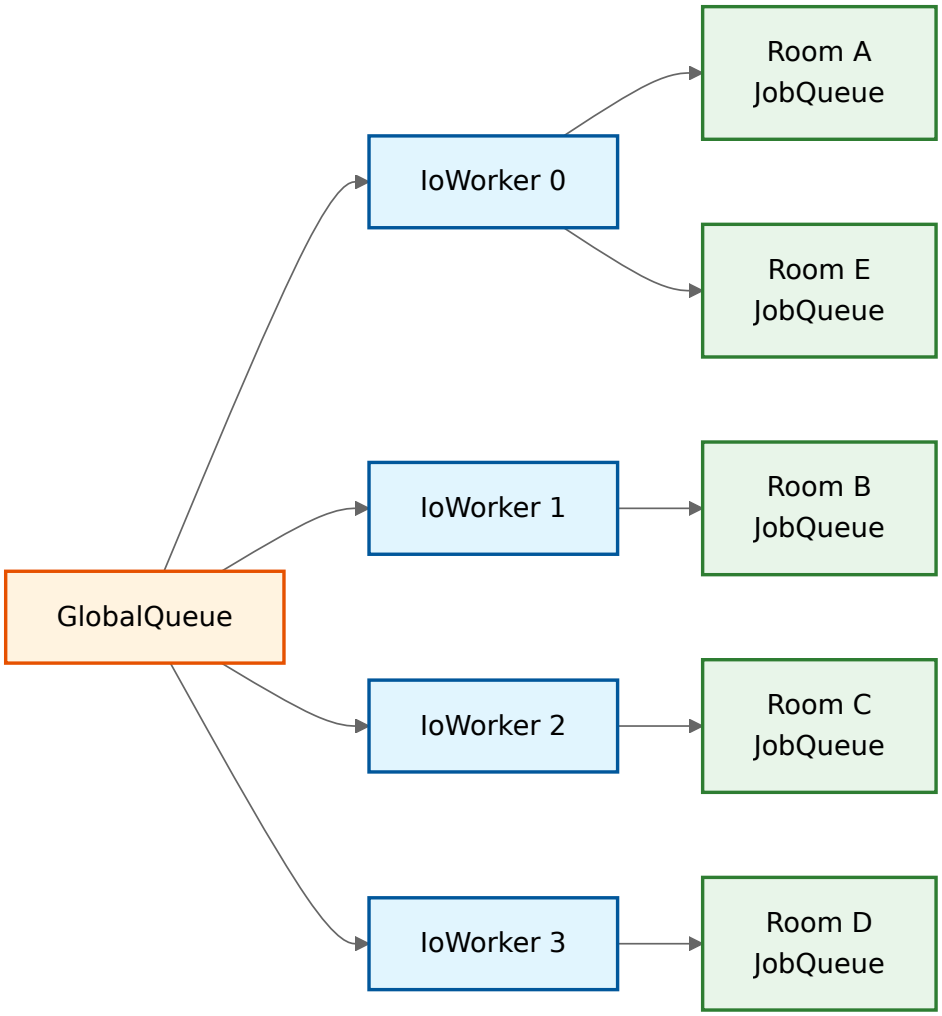

그림 7: GlobalQueue 워크 스틸링

- 어떤 `IoWorker`든 여유가 있으면 `GlobalQueue`에서 `JobQueue`를 가져가므로, Room이 특정 스레드에 고정되지 않고 **부하가 자연스럽게 분산**됩니다.
- 하나의 `JobQueue`는 동시에 하나의 스레드에서만 실행되므로 직렬성이 보장됩니다.

5.1.3 JobTimer — 게임 틱 스케줄링

`JobTimer`는 `steady_clock` 기반의 지연 실행 타이머입니다. Room의 게임 틱(20Hz)이나 버프 지속시간 같은 시간 기반 이벤트를 스케줄링합니다.

- **등록** — `Reserve(delay, callback)`으로 지정 시간 후 실행할 작업을 등록합니다.
- **분배** — `IoWorker`의 Phase 3에서 `DistributeExpired()`가 호출되면, 만료된 타이머의 콜백을 해당 `JobQueue`에 Push합니다. 콜백은 타이머 스레드가 아니라 `JobQueue`의 직렬 실행 흐름에서 수행되므로, 게임 로직과의 동기화 문제가 없습니다.
- **반복 틱** — 게임 틱처럼 주기적인 작업은 콜백 내부에서 자기 자신을 다시 `Reserve`하는 방식으로 반복합니다.

6 PostgreSQL 통합 — 워커 풀 기반 DB 접근

PostgreSQL 쿼리를 `IoWorker` 스레드에서 동기 `libpq`로 바로 호출하면 쿼리 실행이 끝날 때까지 해당 워커가 블로킹되고, 같은 스레드에 배정된 다른 Room들도 영향을 받는 문제가 발생합니다. 이 문제를 해결하기 위해 io_uring의 비동기 DB 접근과 전용 워커 풀 방식 두 가지를 시도해 봤습니다.

첫째는 `PQsendQuery` + `POLL_ADD`로 PostgreSQL 소켓 fd의 readiness를 io_uring에 등록해 결과를 이벤트 루프 안에서 수확하는 방식입니다. 한 번 시도해 봤지만 실질적인 이점이 없었습니다. `POLL_ADD`는 readiness 알림 역할만 하고,

실제 `recv(2)` 는 `PQconsumeInput()` 안에서 libpq가 자체 버퍼로 직접 호출합니다. 그래서 io_uring 고유 최적화인 Provided Buffer Ring이나 Multishot Recv는 DB 경로에 전혀 적용되지 않고, 순수 성능만 보면 `epoll` + 비동기 libpq와 동일한 구조가 됩니다.

둘째는 전용 워커 스레드 풀에 쿼리를 던지고 블로킹 `libpq` 로 실행하는 방식입니다.

상태 머신(`Connecting` / `QuerySent` / `ReadingResult`)과 `POLL_ADD` 재등록 로직이 전부 사라지고 쿼리 하나는 `PQexec` 한 줄이 되며, DB 워커가 블로킹 상태로 오래 머물러도 `IoWorker` 스레드는 영향받지 않아 실패가 격리됩니다. 추후 sqlpp11 같은 동기 ORM 래퍼를 얹더라도 인터페이스가 그대로 맞물립니다.

DB 워커와 콜백 처리

워커 풀 방식에서 남는 문제는 DB 워커가 완료한 콜백을 어느 스레드에서 실행하느냐입니다. DB 워커 스레드에서 그대로 실행하면 Room 상태에 접근하는 순간 데이터 레이스가 발생하므로, 결과를 원래 `IoWorker` 스레드로 돌려보내야 합니다.

`IoRing` 은 `thread_local IoRing* Current()` 로 현재 스레드에 바인딩된 링을 노출합니다. `PgConnectionPool::Execute()` 호출 시점에 그 포인터를 캡처해두면, DB 워커는 `PQexec` 완료 직후 캡처된 링의 `Post(lambda)` 로 콜백을 밀어넣기만 하면 됩니다. `IoWorker` 는 매 루프에서 `ProcessPostedTasks()` 로 이 큐를 비우므로, 콜백은 자연스럽게 원래 스레드에서 실행되고 Room 상태에 락 없이 접근할 수 있습니다.

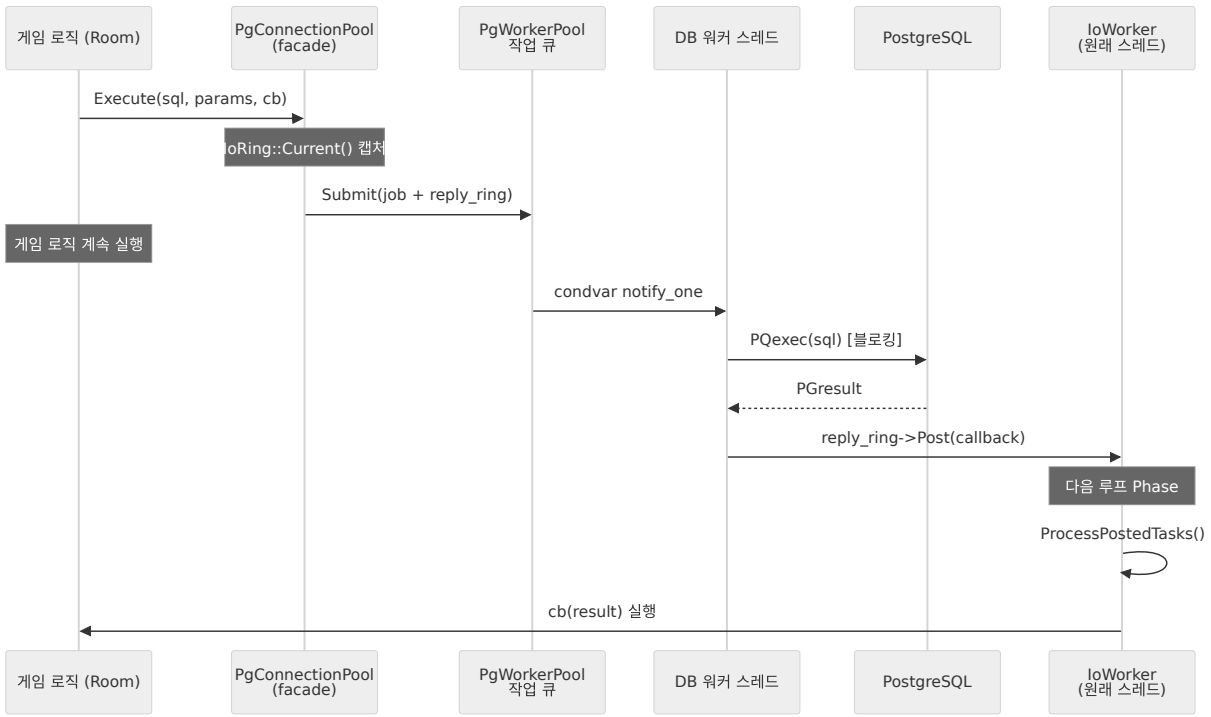

그림 8: 워커 풀 기반 DB 쿼리 흐름

`PgConnectionPool` — 워커-커넥션 1:1 매핑

`PgConnectionPool` 은 얇은 facade이고, 실제 동작은 내부의 `PgWorkerPool` 이 담당합니다. `PgWorkerPool` 은 설정된 `connections_per_worker` 수만큼 스레드를 스폰하고, **각 스레드에 `PgConnection` 하나씩을 1:1로 고정**합니다. 스레드별로 자기 전용 커넥션만 쓰므로 커넥션 접근에 락이 필요 없고, 공유 상태는 `std::mutex` + `std::condition_variable` 로 보호되는 작업 큐 하나뿐입니다. 스레드가 작업을 가져가면 전용 커넥션 위에서 블로킹 `PQexec` 를 실행합니다.

커넥션 풀 자체의 역할은 세 가지입니다.

- **사전 연결** — 서버 시작 시점에 모든 커넥션을 `PQconnectdb`로 열어둡니다. 쿼리가 들어올 때마다 TCP 핸드셰이크와 인증을 반복하는 비용을 피합니다.
- **동시 쿼리 상한** — 워커 스레드 수가 곧 동시 쿼리 상한입니다. 트래픽이 몰려도 PostgreSQL 쪽으로 가는 부하는 이 상한을 넘지 않습니다.
- **백프레셔** — 상한을 초과한 요청은 작업 큐에 쌓여 순서대로 처리됩니다. DB 서버가 수용 가능한 속도 이상으로 요청이 밀려드는 상황을 자연스럽게 흡수합니다.

표 3: 풀 적용 효과

상황	풀 없음	워커 풀
1000명 동시 로그인	1000개 커넥션 시도 → DB 거부	풀 크기(예: 20)만 사용, 나머지는 작업 큐에서 대기
초당 5000 쿼리	5000번 커넥션 생성/해제	20개 커넥션 재사용

7 게임 서버 애플리케이션

서버 코어의 네트워크 엔진, 세션 관리, 브로드캐스트, 비동기 DB 계층 위에 실제 게임 서버 애플리케이션을 구현해 코어의 유효성을 검증했습니다. 클라이언트 측 구현은 별도의 클라이언트 포트폴리오에서 다루며, 이 장에서는 서버 쪽 게임 시스템만 설명합니다.

7.1 프로토콜과 네트워크 동기화

클라이언트와 `[size:uint16][msgId:uint16]` + Protobuf 3 payload 형식의 바이너리 프로토콜로 통신합니다. `PacketBuilder`가 직렬화를 담당하고, 세션별 패킷 핸들러 테이블이 `msgId`를 기준으로 역직렬화와 디스패치를 수행합니다. 이동은 20Hz(`kTickInterval = 50ms`) 주기로 동기화되며, 서버는 각 플레이어의 위치·회전을 같은 Room에 있는 전체 플레이어에게 브로드캐스트합니다.

7.2 게임 시스템

표 4: 게임 시스템 요약

시스템	상세
전투	4종 스킬(Basic Attack / Power Strike / Fireball / Heal), 서버사이드 쿨다운·거리 검증, 데미지 계산 후 `S_DAMAGE` 브로드캐스트
채팅	All / Zone / Party / Whisper 4채널, 채널 타입에 따라 Room 브로드캐스트 / 파티 전송 / 특정 세션 전송
파티	`PartySystem`이 그룹 상태를 관리, 리더 이탈 시 남은 멤버 중 첫 번째에게 자동 위임
인벤토리·재화	`ItemInstance` 기반 슬롯 관리, 3종 재화(Gold/Gems/Tokens), `DbService` 인터페이스로 영속화

전투 판정은 모두 서버에서 수행합니다. 클라이언트가 `C_ATTACK` / `C_FIRE`를 보내면 서버가 쿨다운, 거리, 데미지 계산을 직접 처리합니다. 각 스킬의 파라미터는 `CombatSystem::kSkills` 배열에 정의돼 있고, `skill_cooldowns` 맵에서 클라이언트별 마지막 사용 시각을 검증합니다. 채팅은 채널 타입별로 송신 경로가 다릅니다. 파티/귓속말은 대상

세션을 직접 찾아 전송하고, Zone 채팅과 All 채팅은 같은 Room 내 전체 플레이어에게 브로드캐스트됩니다. 인벤토리와 재화는 `DbService` 추상 인터페이스를 통해 영속화됩니다.

7.3 절차적 던전 생성

120×70 그리드 위에 Cellular Automata로 동굴을 생성하고, 모든 방을 통로로 연결하는 절차적 던전 시스템입니다.

1. **RandomFillMap** — 52% 확률로 벽 배치 (Zone depth에 따라 +2%씩 증가, 최대 62%). 깊은 Zone일수록 좁고 복잡한 동굴이 생성됩니다.
2. **SmoothMap ×5** — 8방향 이웃 벽 수 기준으로 셀을 정리해 유기적 동굴 형태로 수렴합니다.
3. **ProcessMap** — Flood Fill로 영역 식별 후, 15타일 미만 벽 덩어리와 20타일 미만 방을 제거합니다. 작은 벽 기둥은 유지해 동굴 내부 장애물로 활용합니다.
4. **ConnectClosestRooms** — 분리된 방들의 edge tile 쌍에서 최소 거리를 찾아 반지름 1의 좁은 통로로 연결합니다. Main Room에서 모든 방이 도달 가능할 때까지 재귀합니다.
5. **PlacePortals** — 5×5 영역에서 바닥 15칸 이상인 열린 공간에만 배치합니다. 맵 대각선의 8% 이상 간격으로 최대 3개를 분산합니다.
6. **FindSpawnPosition** — Main Room에서 5×5 영역 바닥 타일이 가장 많은 위치(열린 공간 중앙)를 스폰으로 선택합니다.

표 5: 던전 생성 파라미터

파라미터	값	역할
GRID	120×70	맵 크기
FILL	52% (+depth×2)	벽 비율 (깊이 스케일)
WALL_THRESHOLD	15	유지할 최소 벽 크기
ROOM_THRESHOLD	20	유지할 최소 방 크기
PASSAGE_RADIUS	1	좁은 통로

7.4 그래프 기반 Zone 관리

Room 간 연결을 트리가 아닌 **방향 없는 그래프**로 관리합니다. 각 Room은 `connections` 맵(`portal_index → target RoomId`)을 보유하며, 포탈 진입 시 양방향 링크가 생성됩니다.

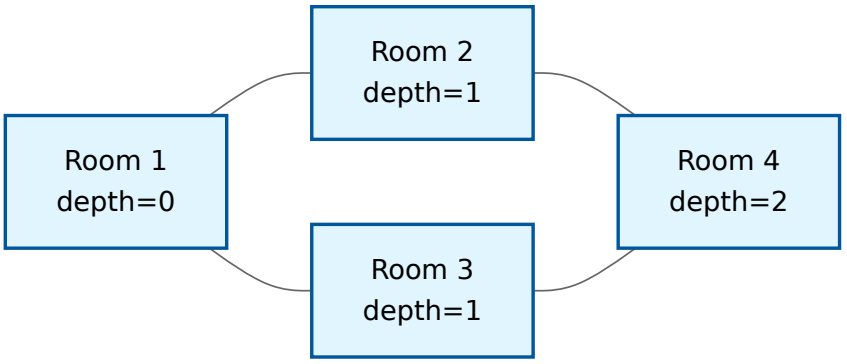

그림 9: 그래프 기반 Zone 연결

포탈을 통한 이동 흐름:

1. 플레이어가 Portal A 진입 → `C_PORTAL(portal_id)` 전송
2. 서버가 `old_room.Connections()[portal_id]` 조회 → 기존 Room이면 이동, 없으면 생성

3. 새 Room 생성 시: 양방향 링크 설정 + `depth = old_depth + 1`
4. 대상 Room에서 역링크 포탈 좌표 근처(±2m)에 플레이어 스폰 → 공간 연속성

7.5 Zone 생명주기

Room은 `shared_ptr`로 관리되며, `empty_since_` 타임스탬프로 비어있는 기간을 추적합니다.

- `AddPlayer` 시 `ClearEmpty()` — 삭제 타이머 초기화
- `RemovePlayer` 후 `players_`가 비면 `MarkEmpty()` — 현재 시각 기록
- 메인 스레드에서 10초마다 `CleanupEmptyRooms()` 실행
- 삭제 조건: 비어있음 + 30초 경과 + 루트 Room이 아님
- 삭제 시 모든 Room의 `connections`에서 해당 Room 참조 제거

이 설계로 포탈 전환 중 비동기 `AddPlayer`가 완료되기 전에 Room이 삭제되는 race condition을 방지합니다.

7.6 DB 영속화와 백엔드 교체

GameServer의 기본 DB 백엔드는 6장에서 설명한 `PgConnectionPool` 워커 풀 위에 올린 **PostgreSQL**입니다. 계정·캐릭터·인벤토리·재화가 모두 PostgreSQL에 영속화되며, 쿼리는 DB 워커 스레드에서 블로킹 `libpq`로 실행된 뒤 결과 콜백이 `IoRing::Post`로 원래의 `IoWorker` 스레드로 돌아옵니다. 라이브러리 계층의 DB 기능이 애플리케이션 계층에서 실제로 소비된다는 것을 이 경로가 증명합니다.

동시에 GameServer는 `DbService` 추상 인터페이스를 두고 세 가지 구현체(`PgDbService` / `SqliteDbService` / `MemoryDbService`)가 같은 인터페이스 뒤에 교체 가능하도록 설계했습니다. 어느 쪽을 사용할지는 환경변수 `GAMESERVER_DB`로 런타임에 결정됩니다.

표 6: DbService 백엔드 옵션

백엔드	선택 방법	용도
PostgreSQL (기본)	`./GameServer`	운영 환경. 6장의 `PgConnectionPool` 워커 풀 위에서 동작
SQLite	`GAMESERVER_DB=sqlite ./GameServer`	오프라인/로컬 개발. sqlite3 amalgamation을 vendored로 포함
In-Memory	테스트용	프로세스 수명 동안만 유지, 유닛 테스트

7.6.1 PgDbService — 워커 풀 위의 DbService 어댑터

`PgDbService`는 6장의 `PgConnectionPool`을 감싼 얇은 어댑터입니다. 각 메서드는 SQL 쿼리를 워커 풀에 제출하고, `PgResult` → 도메인 구조체(`CharInfo` / `ItemData` / `CurrencyData`) 매핑을 수행한 뒤 사용자 콜백으로 넘깁니다. 쿼리 실행은 DB 워커 스레드에서 이뤄지지만, 결과 콜백은 `IoRing::Post` 덕분에 원래의 `IoWorker` 스레드로 되돌아와 실행되므로 핸들러 측 코드는 별도의 동기화 없이 Room 상태에 접근할 수 있습니다.

체인이 필요한 쿼리(예: `Login`의 "계정 조회 → 없으면 자동 등록", `CreateChar`의 "캐릭터 insert → 기본 재화 seed")는 첫 콜백 안에서 두 번째 `pool->Execute()`를 호출하는 방식으로 구성합니다. 두 콜백 모두 같은 `IoWorker` 스레드에서 실행되므로, 체인 중간의 상태 변수를 별도 동기화 없이 캡처해 다음 단계로 넘길 수 있습니다.

스키마 DDL은 생성 시점에 짧은 수명의 bootstrap `PgConnection`을 하나 열어 동기로 실행합니다. main 스레드에는 아직 `IoRing::Current()`가 없어서 워커 풀을 통해 DDL을 제출할 수 없기 때문입니다. 생성자가 완료되면

bootstrap 커넥션은 즉시 파괴되고, 이후의 모든 쿼리는 워커 풀을 거칩니다.

7.7 독텍 루팅 시스템

사망 시 사망 위치에 독텍(Dog Tag) 아이템이 스폰되어 바닥에 남습니다. 플레이어가 3m 이내에서 `C_PICKUP` 패킷을 전송하면 서버가 거리 검증 후 인벤토리에 추가합니다.

- `GroundItem` 구조체: ground_id, item_def_id, 위치, 라벨, 15초 수명
- `S_GROUND_ITEM_SPAWN` / `S_GROUND_ITEM_DESPAWN`: 모든 클라이언트에 브로드캐스트
- `TryPickup()`: 거리 검증 → `S_ITEM_ADD` + `S_GROUND_ITEM_DESPAWN`
- Room::OnTick에서 수명 소진된 아이템 자동 제거
- 플레이어 킬만 독텍 드롭 (봇끼리의 킬은 드롭 없음)

7.8 세션 레벨 패킷 라우팅

`C_PORTAL` 처럼 Room 외부에서 처리해야 하는 패킷을 위해 `OnPacket` (최상위 라우터)에서 상태와 무관하게 세션 레벨 패킷을 먼저 분기합니다.

game_session.cpp — OnPacket

```
void GameSession::OnPacket(uint16_t msg_id, const std::byte* data, uint32_t len) {
    switch (static_cast<MsgId>(msg_id)) {
        case MsgId::C_PORTAL: handler::HandlePortal(*this, data, len); return;
        default: break;
    }
    if (state_ == SessionState::InRoom) HandleInRoomPacket(msg_id, data, len);
    else HandlePreRoomPacket(msg_id, data, len);
}
```

이 구조로 Room의 `HandlePacket` 과 세션 레벨 로직이 깔끔하게 분리되며, 향후 다른 세션 레벨 패킷이 추가될 때 같은 switch에 넣으면 됩니다.

8 벤치마크 — io_uring vs epoll

ServerCore(io_uring)가 기존 epoll 대비 실제로 어떤 차이를 보이는지 검증하기 위해, 동일한 게임 로직을 두 I/O 백엔드로 구현해 비교했습니다.

8.1 테스트 환경

- **환경**: WSL2 Ubuntu 24.04, localhost, Intel i7 (8코어)
- **도구**: `game_bench` / `echo_bench` (`tools/run_bench.sh`)
- **게임 로직**: Login → EnterGame → Move(20Hz) → Attack(2Hz, 4종 스킬), 두 서버 동일

표 7: 벤치마크 서버 구성

서버	I/O 백엔드	설명
io_uring	io_uring	IoWorker가 I/O 디스패치와 Zone 틱을 단일 루프에서 처리, ServerCore 사용
Epoll	epoll	베이스라인 epoll 구현, ServerCore 미사용

8.2 Echo 벤치마크

게임 로직 없이 io_uring 코어의 순수 I/O 성능만 측정했습니다. 64바이트 페이로드, pipeline=1 조건입니다.

표 8: Echo 벤치마크 결과

클라이언트	처리량 (echo/s)	p50 (µs)	p99 (µs)
10	218,498	30	157
50	305,183	152	367
100	326,913	293	594

100 클라이언트에서 최대 처리량 **327K echo/s**, p50 레이턴시 **293µs**를 기록해 서브 밀리초 수준의 응답성을 확인했습니다.

8.3 브로드캐스트 지연 — 단일 존

`C_MOVE` 수신 → `S_MOVE` 브로드캐스트 완료까지의 지연(µs)입니다. 모든 봇이 하나의 존에 몰린 조건입니다.

표 9: 브로드캐스트 p50 지연 (단일 존, µs)

봇 수	구성	io_uring p50	Epoll p50
40	4스레드	838	3,322
200	4스레드	10,001	4,419,583
400	8스레드	3,025	—

- **40봇**: io_uring이 서브 밀리초, Epoll은 3ms 수준
- **200봇**: Epoll이 뮤텍스 경합으로 초 단위에 진입, io_uring은 10ms 수준 유지
- **400봇**: io_uring은 여전히 3ms 수준을 유지, Epoll은 서버 포화로 측정 불가

8.4 Tail Latency — p99

표 10: Tail latency p99 (µs)

봇 수	구성	io_uring p99	Epoll p99
40	1스레드	1,887	6,067
200	4스레드	24,191	11,794,952

200봇 p99 기준, io_uring은 24ms 수준을 유지한 반면 Epoll은 11.8초까지 증가했습니다.

8.5 처리량

`S_MOVE + S_ATTACK + S_DAMAGE` 수신 메시지/초 기준입니다.

표 11: 단일 존 처리량

봇 수	구성	io_uring (msg/s)	Epoll (msg/s)	배율
40	1스레드	37,288	37,288	1.0×
200	4스레드	950,656	523,846	**1.8×**

소규모(40봇)에서는 차이가 없지만, 200봇부터 Epoll이 뮤텍스 경합으로 처리량이 제한되기 시작합니다.

8.6 멀티 Room — Room 샤딩 효과

봇을 여러 Room으로 분산하면 Room당 약 20명이 되어, 단일 존의 $O(N^2)$ 브로드캐스트 병목이 해소됩니다.

표 12: 멀티 Room 브로드캐스트 p50

규모	구성	io_uring p50 (µs)	Epoll p50 (µs)
200봇 / 10 Room	4T×50C	846	897
400봇 / 20 Room	4T×100C	920	691

Room 샤딩을 적용하면 두 서버 모두 **서브 밀리초** 수준으로 수렴합니다. 실제 게임에서는 Room 분산이 기본이므로, io_uring의 이점은 대규모 단일 존에서 두드러지게 드러납니다.

8.7 결론

항목	io_uring	Epoll
강점	400봇까지 ms 수준 지연 유지, 배치 제출로 시스템콜 최소화	소규모에서 구현이 단순, SQE 오버헤드 없음
병목	Zone 틱이 IO와 스레드 공유	200봇 이상에서 브로드캐스트 뮤텍스 경합

두 백엔드의 차이는 단일 존에 봇이 몰릴수록 뚜렷해집니다. 40봇 구간에서는 차이가 크지 않지만, 200봇을 넘어서면 epoll은 브로드캐스트 경로의 뮤텍스 경합으로 p99가 초 단위까지 벌어지는 반면, io_uring 쪽은 400봇까지도 밀리초대를 유지합니다. 반대로 Room 샤딩으로 부하를 분산하면 두 서버 모두 서브 밀리초로 수렴해 차이가 사라지는데, 이는 io_uring의 이점이 “작은 방이 많은” 상황이 아니라 “한 방에 사람이 많은” 상황에서 드러난다는 것을 보여줍니다.